CAP4770 Project

Predicting Flight Delays

Group Project:

Noah Spataro, Gareth Pace, Luiz Feitosa

mySQLsetup.sql

Code Blame 21 lines (20 loc) · 490 Bytes

```sql
CREATE DATABASE IF NOT EXISTS cap4770_project;
USE cap4770_project;

CREATE TABLE IF NOT EXISTS flights (
    id INT AUTO_INCREMENT PRIMARY KEY,
    YEAR INT,
    MONTH INT,
    DAY_OF_WEEK INT,
    OP_UNIQUE_CARRIER VARCHAR(10),
    ORIGIN_AIRPORT_ID INT,
    DEST_AIRPORT_ID INT,
    CRS_DEP_TIME VARCHAR(4),
    DEP_DELAY FLOAT,
    TAXI_OUT FLOAT,
    CANCELLED TINYINT,
    DIVERTED TINYINT,
    CRS_ELAPSED_TIME FLOAT,
    AIR_TIME FLOAT,
    DISTANCE FLOAT,
    DISTANCE_GROUP INT
);
```

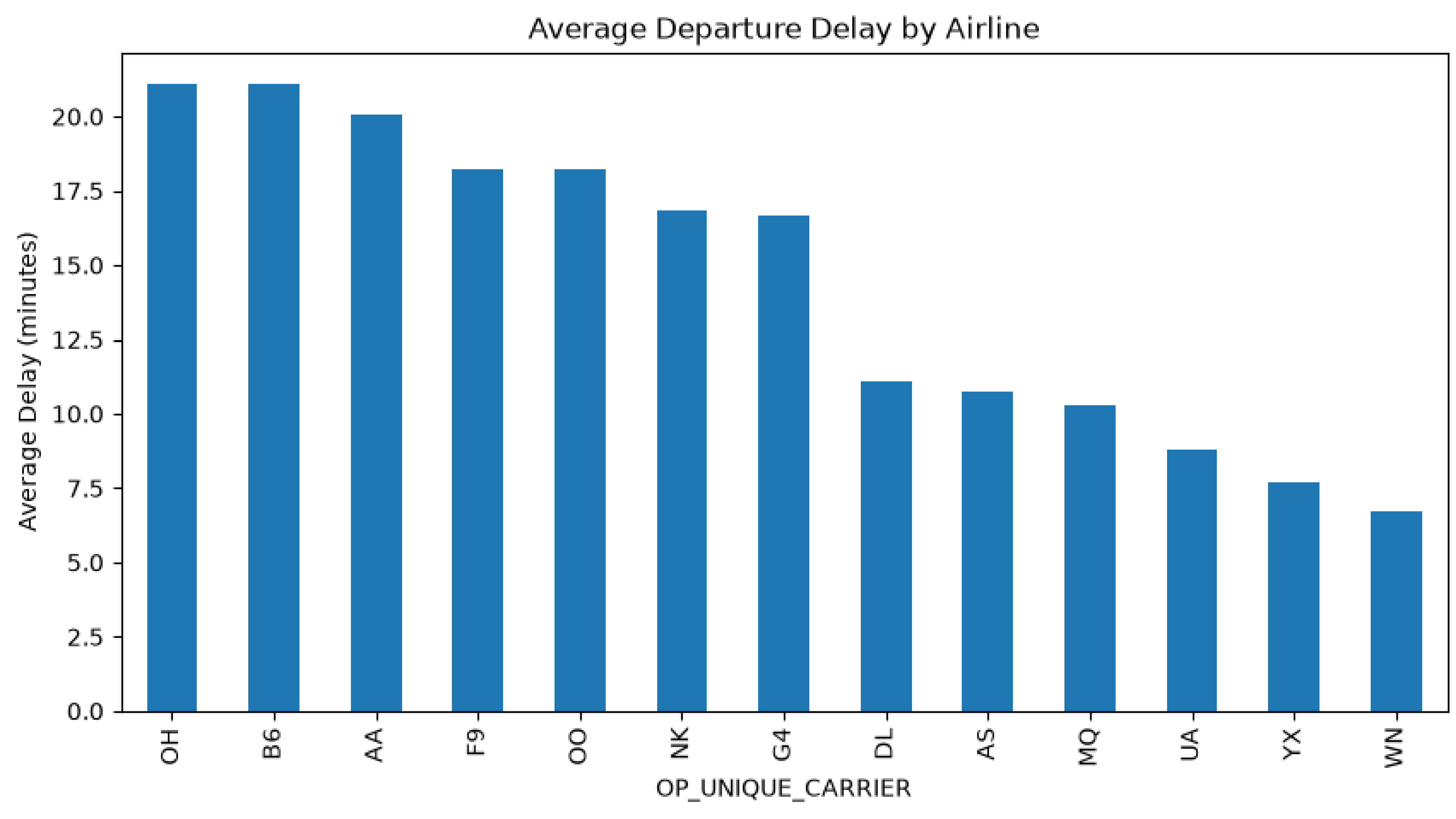
Project.ipynb

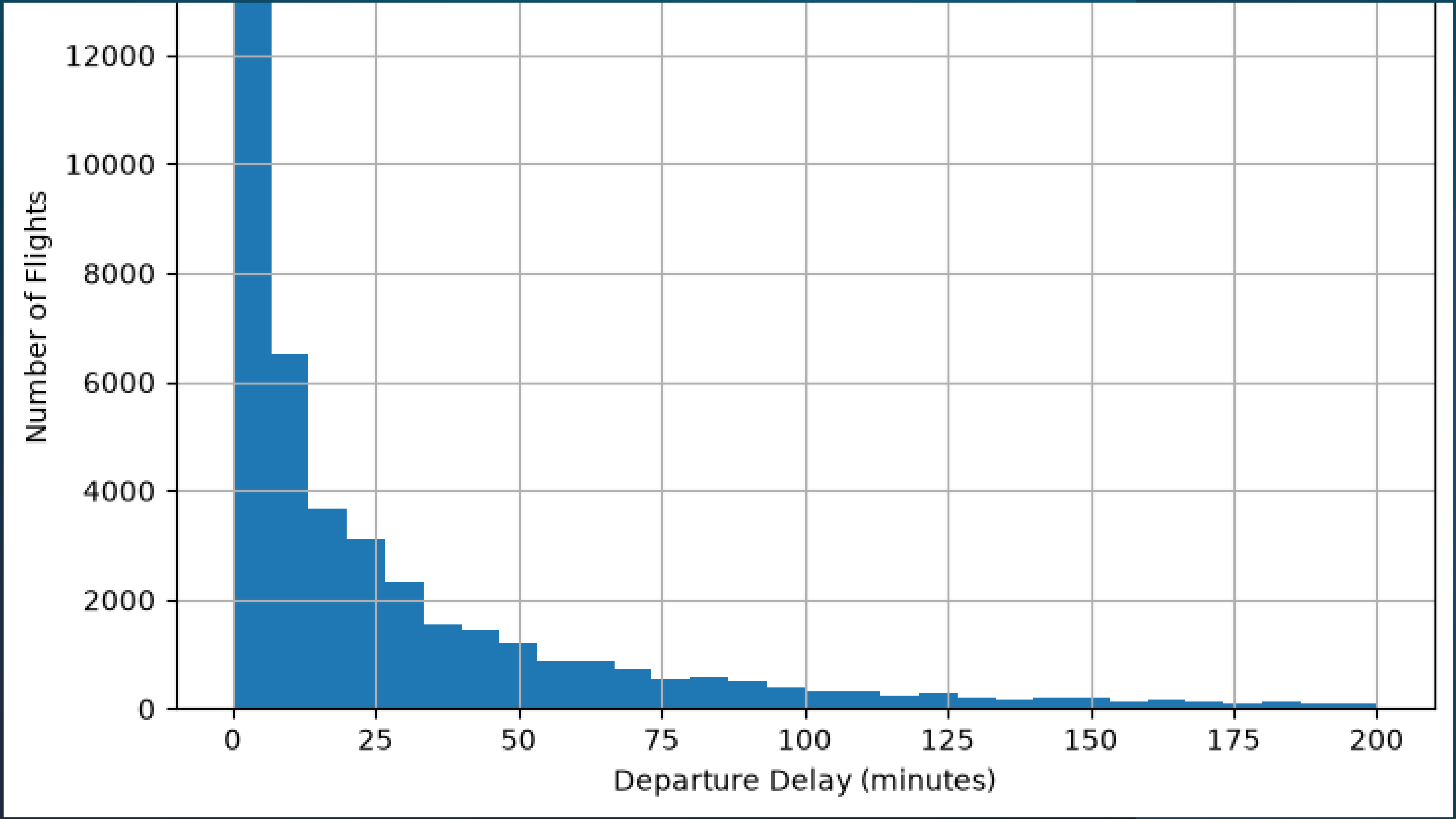
In [ ]:

```
# Loads the flight dataset (data.csv) into the 'flights' table of the
# 'cap4770_project' MySQL database
import pandas as pd
from getpass import getpass
from sqlalchemy import create_engine

password = getpass("MySQL password: ")
# Connects to the local database as 'root'
engine = create_engine("mysql+pymysql://root:" + password + "@localhost/cap4770_project")

df = pd.read_csv("data.csv", dtype={"CRS_DEP_TIME": str})

# Appends the rows to the 'flights' table.
df.to_sql("flights", engine, if_exists="append", index=False, chunksize=1000)

print("Done! Loaded", len(df), "rows into MySQL.")
```

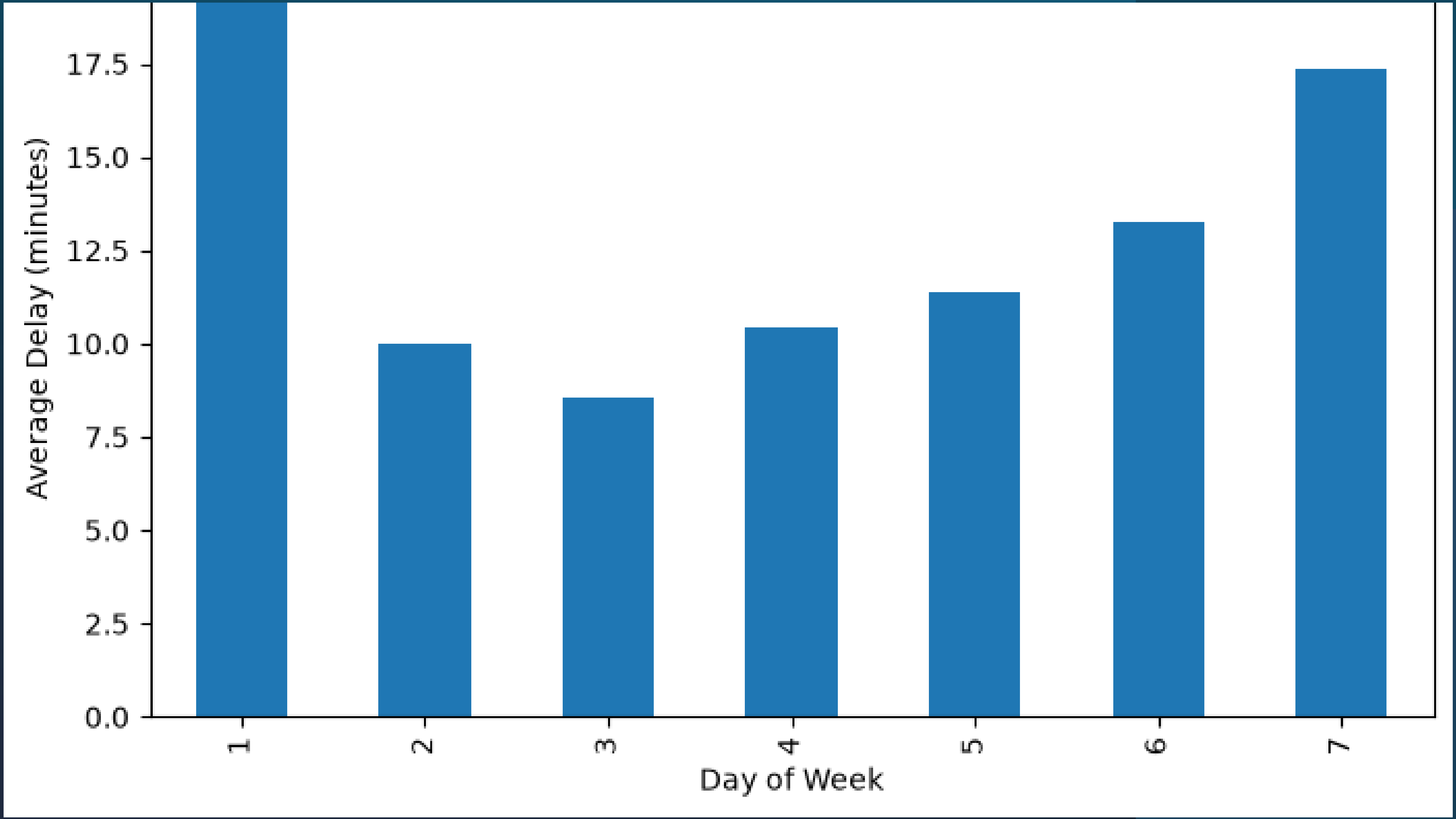
In [1]:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

df = pd.read_csv(
    "data.csv",
    dtype={"CRS_DEP_TIME": str}
)


#This code filters the dataset down to 20% of the original (around 100,000 entries out of 500,000)
#Flights are considered delayed if they are delayed by more than 15 minutes
df["Delayed"] = (df["DEP_DELAY"] > 15).astype(int)

#The proportion of delayed flights is preserved
sample, remainder = train_test_split(
    df,
    train_size=0.20,
    stratify=df["Delayed"],
    random_state=42
)

df = sample.copy()

#Preprocessing starts here
#Remove cancelled and diverted flights; they don't impact delay values
df = df[(df["CANCELLED"] == 0) & (df["DIVERTED"] == 0)].copy()
#Scale numeric values where necessary
scaler = StandardScaler()

numeric = [
    "DISTANCE",
    "CRS_ELAPSED_TIME",
]

df[numeric] = scaler.fit_transform(df[numeric])

#Get departure hour
df["CRS_DEP_TIME"] = df["CRS_DEP_TIME"].str.zfill(4)
df["DEP_HOUR"] = df["CRS_DEP_TIME"].str[:2].astype(int)

df_clean = df.copy()

#Make a graph showing average delay by airline
avg_delay = (
    df.groupby("OP_UNIQUE_CARRIER")["DEP_DELAY"]
      .mean()
      .sort_values(ascending=False)
)

avg_delay.plot(
    kind="bar",
    figsize=(10,5),
    title="Average Departure Delay by Airline"
)

plt.ylabel("Average Delay (minutes)")
plt.show()

#Make a graph showing distribution of delays
#Negative delays (early departures) and delays greater than 200 are not included to improve visualization
delay = df[(df["DEP_DELAY"] >= 0) & (df["DEP_DELAY"] <= 200)]

plt.figure(figsize=(8,5))

delay["DEP_DELAY"].hist(bins=30)

plt.title("Distribution of Delays")
plt.xlabel("Departure Delay (minutes)")
plt.ylabel("Number of Flights")
plt.show()

#Make a graph showing average delay for each day of the week
avg_day = df.groupby("DAY_OF_WEEK")["DEP_DELAY"].mean()

avg_day.plot(kind="bar", figsize=(8,5))
plt.title("Average Departure Delay by Day of Week")
plt.xlabel("Day of Week")
plt.ylabel("Average Delay (minutes)")
plt.show()
```


Average Departure Delay by Airline
Average Delay (minutes)
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
OH
B6
AA
F9
OO
NK
G4
DL
AS
MQ
UA
YX
WN
OP_UNIQUE_CARRIER



12000
10000
8000
6000
4000
2000
0
Number of Flights
0
25
50
75
100
125
150
175
200
Departure Delay (minutes)



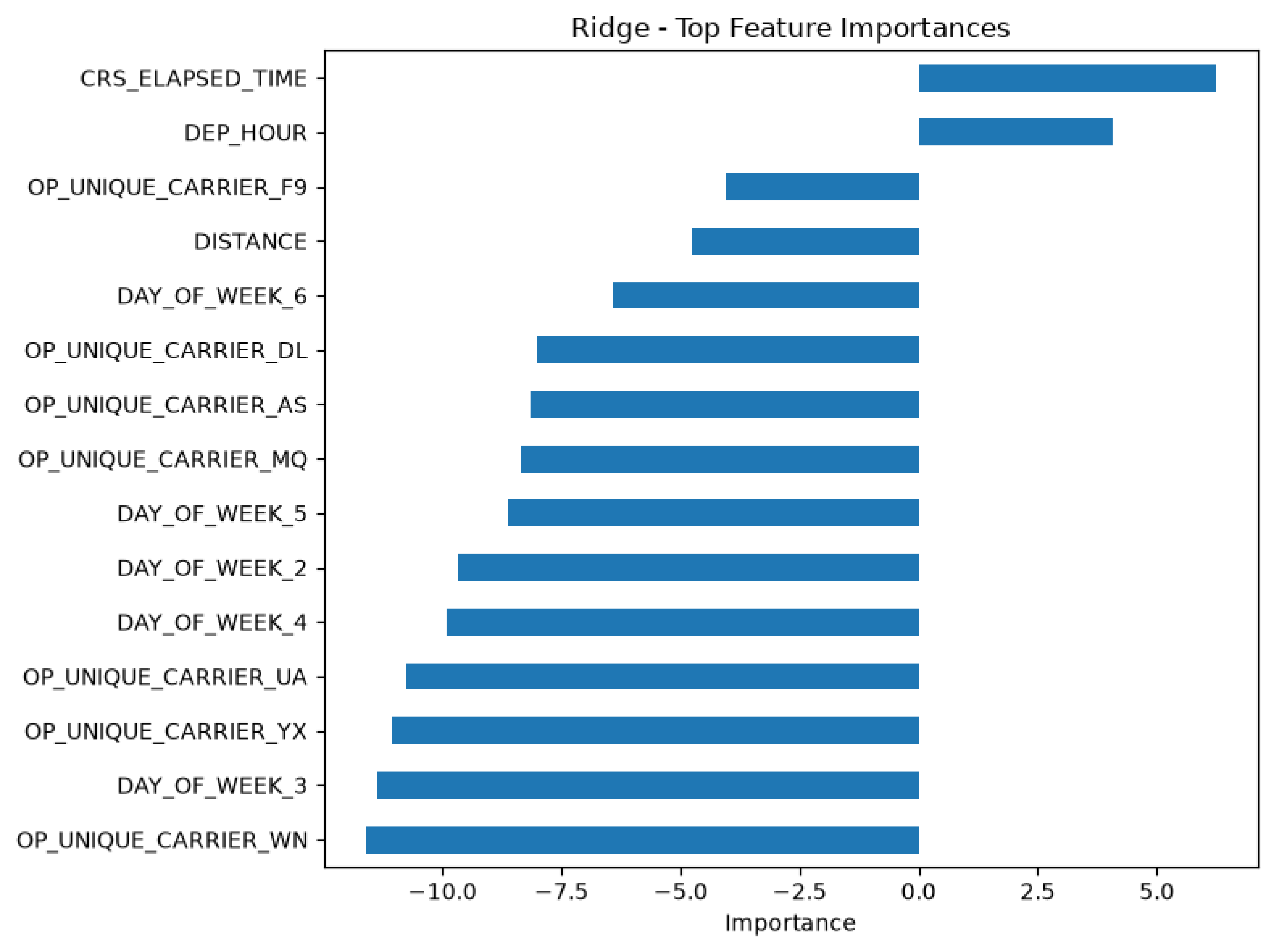
Average Delay (minutes)
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
1
2
3
4
5
6
7
Day of Week

Project.ipynb

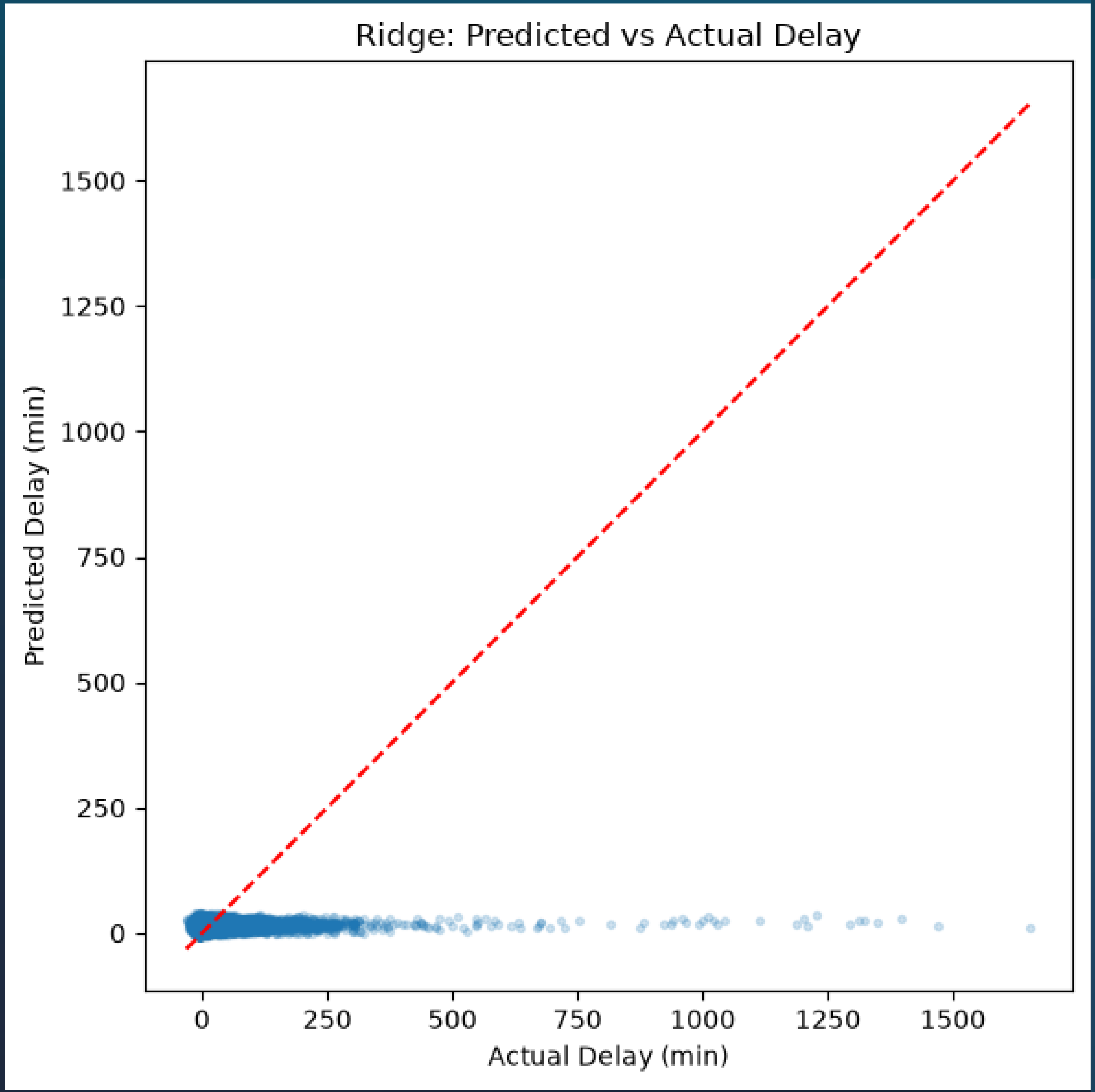
In [2]:

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, GridSearchCV, KFold, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

RANDOM_STATE = 42

#Reuse the cleaned 20% sample
model_df = df_clean.copy()

#Features/Target
TARGET = "DEP_DELAY"
numeric_features = ["DISTANCE", "CRS_ELAPSED_TIME", "DEP_HOUR"]
categorical_features = ["OP_UNIQUE_CARRIER", "DAY_OF_WEEK", "MONTH"]

model_cols = numeric_features + categorical_features + [TARGET]
data = model_df[model_cols].dropna()

X = pd.get_dummies(data[numeric_features + categorical_features],
                   columns=categorical_features, drop_first=True)
y = data[TARGET]

#Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE
)

scaler = StandardScaler()
X_train[numeric_features] = scaler.fit_transform(X_train[numeric_features])
X_test[numeric_features] = scaler.transform(X_test[numeric_features])

#Cross validation on training-set
cv = KFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

candidates = {
    "LinearRegression": LinearRegression(),
    "Ridge": Ridge(alpha=1.0, random_state=RANDOM_STATE),
    "RandomForestRegressor": RandomForestRegressor(
        n_estimators=200, max_depth=12, random_state=RANDOM_STATE, n_jobs=-1
    ),
}

cv_scores = {}
print("5-fold CV RMSE on training data (model selection, no test-set peeking):")
for name, model in candidates.items():
    scores = cross_val_score(model, X_train, y_train, cv=cv,
                             scoring="neg_root_mean_squared_error", n_jobs=-1)
    rmse = -scores.mean()
    cv_scores[name] = rmse
    print(f"  {name}: {rmse:.3f} (+/- {scores.std():.3f})")

best_name = min(cv_scores, key=cv_scores.get)
print(f"\nSelected model (lowest CV RMSE): {best_name}")

#Hyperparameter tuning for model using training-set
if best_name == "RandomForestRegressor":
    param_grid = {"n_estimators": [100, 200], "max_depth": [8, 12, None]}
    grid = GridSearchCV(
        RandomForestRegressor(random_state=RANDOM_STATE, n_jobs=-1),
        param_grid, cv=cv, scoring="neg_root_mean_squared_error", n_jobs=-1,
    )
elif best_name == "Ridge":
    param_grid = {"alpha": [0.1, 1.0, 10.0, 50.0]}
    grid = GridSearchCV(
        Ridge(random_state=RANDOM_STATE), param_grid, cv=cv,
        scoring="neg_root_mean_squared_error",
    )
else:
    grid = None

if grid is not None:
    grid.fit(X_train, y_train)
    print(f"Best params: {grid.best_params_}")
    print(f"Best CV RMSE after tuning: {-grid.best_score_:.3f}")
    best_model = grid.best_estimator_
else:
    best_model = candidates[best_name]
    best_model.fit(X_train, y_train)

#Final eval with test set
final_preds = best_model.predict(X_test)
mae = mean_absolute_error(y_test, final_preds)
mse = mean_squared_error(y_test, final_preds)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, final_preds)

print(f"\nFinal held-out test performance ({best_name}):")
print(f"  MAE:  {mae:.3f}")
print(f"  MSE:  {mse:.3f}")
print(f"  RMSE: {rmse:.3f}")
print(f"  R^2:  {r2:.3f}")

#Feature importances
if hasattr(best_model, "feature_importances_"):
    importances = pd.Series(best_model.feature_importances_, index=X.columns)
    importances = importances.sort_values(ascending=False).head(15)
elif hasattr(best_model, "coef_"):
    importances = pd.Series(best_model.coef_, index=X.columns)
    importances = importances.reindex(
        importances.abs().sort_values(ascending=False).index
    ).head(15)
else:
    importances = None

if importances is not None:
    plt.figure(figsize=(8, 6))
    importances.sort_values().plot(kind="barh")
    plt.title(f"{best_name} - Top Feature Importances")
    plt.xlabel("Importance")
    plt.tight_layout()
    plt.savefig("regression_feature_importance.png")
    print("\nSaved regression_feature_importance.png")

#Predicted vs. actual plotted
plt.figure(figsize=(6, 6))
plt.scatter(y_test, final_preds, alpha=0.2, s=8)
lims = [min(y_test.min(), final_preds.min()), max(y_test.max(), final_preds.max())]
plt.plot(lims, lims, "r--")
plt.xlabel("Actual Delay (min)")
plt.ylabel("Predicted Delay (min)")
plt.title(f"{best_name}: Predicted vs Actual Delay")
plt.tight_layout()
plt.savefig("regression_pred_vs_actual.png")
print("Saved regression_pred_vs_actual.png")
```

```
5-fold CV RMSE on training data (model selection, no test-set peeking):
  LinearRegression: 58.800 (+/- 2.475)
  Ridge: 58.800 (+/- 2.475)
  RandomForestRegressor: 59.415 (+/- 2.463)

Selected model (lowest CV RMSE): Ridge
Best params: {'alpha': 50.0}
Best CV RMSE after tuning: 58.799

Final held-out test performance (Ridge):
  MAE:  25.035
  MSE:  3583.443
  RMSE: 59.862
  R^2:  0.015

Saved regression_feature_importance.png
Saved regression_pred_vs_actual.png
```



Ridge - Top Feature Importances
CRS_ELAPSED_TIME
DEP_HOUR
OP_UNIQUE_CARRIER_F9
DISTANCE
DAY_OF_WEEK_6
OP_UNIQUE_CARRIER_DL
OP_UNIQUE_CARRIER_AS
OP_UNIQUE_CARRIER_MQ
DAY_OF_WEEK_5
DAY_OF_WEEK_2
DAY_OF_WEEK_4
OP_UNIQUE_CARRIER_UA
OP_UNIQUE_CARRIER_YX
DAY_OF_WEEK_3
OP_UNIQUE_CARRIER_WN
−10.0
−7.5
−5.0
−2.5
0.0
2.5
5.0
Importance


Ridge: Predicted vs Actual Delay
Predicted Delay (min)
Actual Delay (min)
0
250
500
750
1000
1250
1500
0
250
500
750
1000
1250
1500

Project.ipynb

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

RANDOM_STATE = 42

# Reuse the cleaned 20% sample from the regression step
clf_df = df_clean.copy()

# Features/Target
CLF_TARGET = "Delayed"
clf_numeric_features = ["DISTANCE", "CRS_ELAPSED_TIME", "DEP_HOUR"]
clf_categorical_features = ["OP_UNIQUE_CARRIER", "DAY_OF_WEEK", "MONTH"]

clf_model_cols = clf_numeric_features + clf_categorical_features + [CLF_TARGET]
clf_data = clf_df[clf_model_cols].dropna()

X_clf = pd.get_dummies(clf_data[clf_numeric_features + clf_categorical_features],
                        columns=clf_categorical_features, drop_first=True)
y_clf = clf_data[CLF_TARGET]

# Stratified train/test split
X_clf_train, X_clf_test, y_clf_train, y_clf_test = train_test_split(
    X_clf, y_clf, test_size=0.2, stratify=y_clf, random_state=RANDOM_STATE
)

clf_scaler = StandardScaler()
X_clf_train[clf_numeric_features] = clf_scaler.fit_transform(X_clf_train[clf_numeric_features])
X_clf_test[clf_numeric_features] = clf_scaler.transform(X_clf_test[clf_numeric_features])

print("Train class balance:")
print(y_clf_train.value_counts(normalize=True))
print("\nTest class balance:")
print(y_clf_test.value_counts(normalize=True))
```

```
Train class balance:
Delayed
0    0.800476
1    0.199524
Name: proportion, dtype: float64

Test class balance:
Delayed
0    0.800454
1    0.199546
Name: proportion, dtype: float64
```

Project.ipynb

```python
from sklearn.linear_model import LogisticRegression

# Train a simple logistic regression to classify delayed vs on-time flights
clf_model = LogisticRegression(class_weight="balanced", max_iter=1000, random_state=RANDOM_STATE)
clf_model.fit(X_clf_train, y_clf_train)

# Predict on the test set
clf_preds = clf_model.predict(X_clf_test)
```

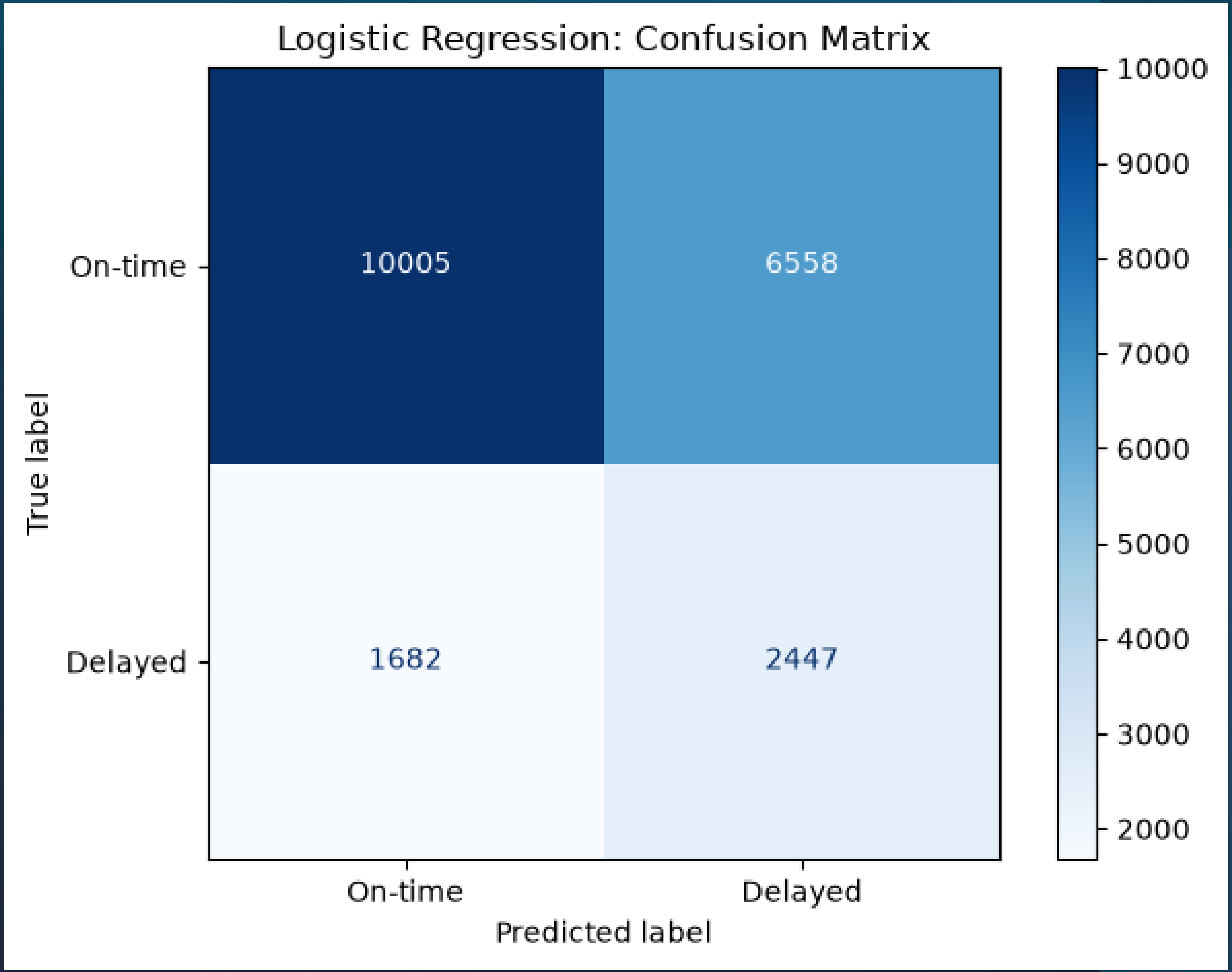
```python
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report
)

accuracy = accuracy_score(y_clf_test, clf_preds)
precision = precision_score(y_clf_test, clf_preds)
recall = recall_score(y_clf_test, clf_preds)
f1 = f1_score(y_clf_test, clf_preds)
cm = confusion_matrix(y_clf_test, clf_preds)

print(f"Accuracy:  {accuracy:.3f}")
print(f"Precision: {precision:.3f}")
print(f"Recall:    {recall:.3f}")
print(f"F1-score:  {f1:.3f}")
print("\nConfusion matrix:")
print(cm)
print("\n" + classification_report(y_clf_test, clf_preds, target_names=["On-time", "Delayed"]))
```

```
Accuracy:  0.602
Precision: 0.272
Recall:    0.593
F1-score:  0.373

Confusion matrix:
[[10005  6558]
 [ 1682  2447]]

              precision    recall  f1-score   support

     On-time       0.86      0.60      0.71     16563
     Delayed       0.27      0.59      0.37      4129

    accuracy                           0.60     20692
   macro avg       0.56      0.60      0.54     20692
weighted avg       0.74      0.60      0.64     20692
```

Project.ipynb

In [6]:

```
from sklearn.metrics import ConfusionMatrixDisplay

# Visualize how many delayed/on-time flights were predicted correctly
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["On-time", "Delayed"])
disp.plot(cmap="Blues", values_format="d")
plt.title("Logistic Regression: Confusion Matrix")
plt.tight_layout()
plt.show()
```


Logistic Regression: Confusion Matrix
On-time
Delayed
True label
10005
6558
1682
2447
On-time
Delayed
Predicted label
10000
9000
8000
7000
6000
5000
4000
3000
2000